Name : Vatsal Postiwala PRN No: 26070122509 Batch : B3(2025-2029)

ASSIGNMENT No: 25

TITLE: Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Problem Statement

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives

To create and manage multiple threads using the Thread class.

Theory

A thread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the run() method. A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as start(), sleep(), join(), and interrupt() control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications.

Code:

```
package E25;

import java.lang.*;

class MyThread extends Thread {
   public void run() {
      System.out.println(Thread.currentThread().getName() + " is running");

      try {
         Thread.sleep(1000);
      } catch (InterruptedException e) {
         System.out.println("Thread interrupted");
      }
```

```
        System.out.println(Thread.currentThread().getName() + " has finished");
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        System.out.println("State of t1 before start: " + t1.getState());

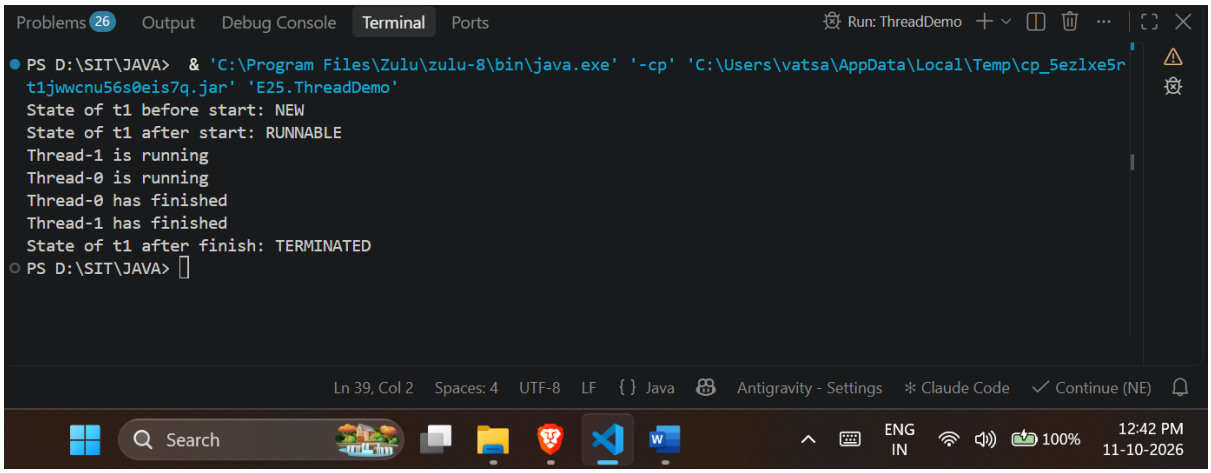
        t1.start();
        t2.start();

        System.out.println("State of t1 after start: " + t1.getState());

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted");
        }

        System.out.println("State of t1 after finish: " + t1.getState());
    }
}
```

Output:



Exercise

1. Create a program where multiple threads print numbers simultaneously.

Code:

```
package E25;

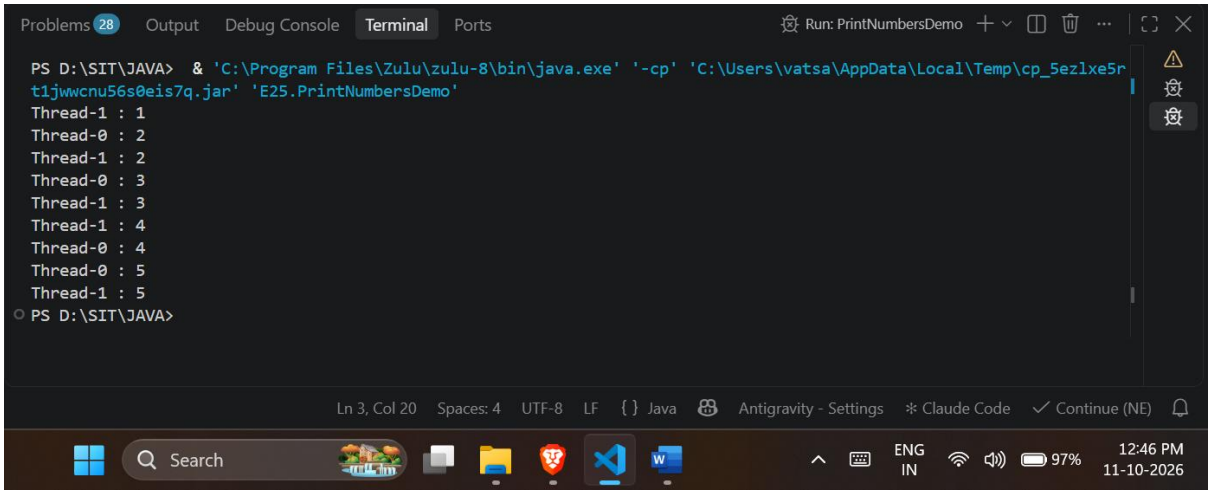
import java.lang.*;

class NumberThread extends Thread {
   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(Thread.currentThread().getName() + " : " + i);
         try {
            Thread.sleep(500);
         } catch (InterruptedException e) {
            System.out.println("Interrupted");
         }
      }
   }
}

public class PrintNumbersDemo {
   public static void main(String[] args) {
      NumberThread t1 = new NumberThread();
      NumberThread t2 = new NumberThread();

      t1.start();
      t2.start();
   }
}
```

Output:



2. Create a Java program using **multiple threads** to simulate bank transactions. One thread performs **deposit operations**, and another performs **withdrawal operations**, both printing transaction details simultaneously.

Code:

```
package E25;

import java.lang.*;

class BankAccount {
  int balance = 1000;

  public synchronized void deposit(int amount) {
    balance += amount;
    System.out.println("Deposited: " + amount + "  Balance: " + balance);
  }

  public synchronized void withdraw(int amount) {
    if (balance >= amount) {
      balance -= amount;
      System.out.println("Withdrawn: " + amount + "  Balance: " + balance);
    } else {
      System.out.println("Insufficient balance");
    }
  }
}
```

```
class DepositThread extends Thread {
  BankAccount account;

  public DepositThread(BankAccount account) {
    this.account = account;
  }

  public void run() {
    for (int i = 0; i < 3; i++) {
      account.deposit(200);
      try {
        Thread.sleep(400);
      } catch (InterruptedException e) {}
    }
  }
}

class WithdrawThread extends Thread {
  BankAccount account;

  public WithdrawThread(BankAccount account) {
    this.account = account;
  }

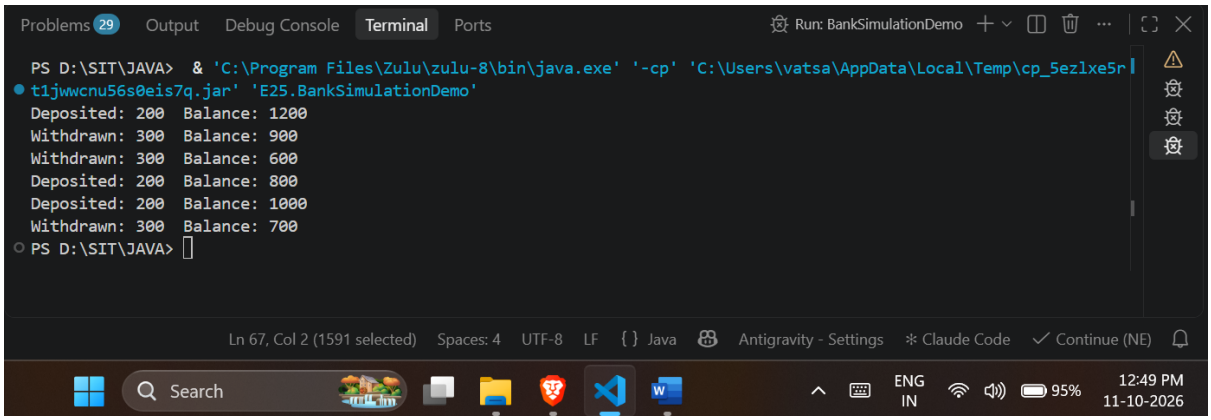
  public void run() {
    for (int i = 0; i < 3; i++) {
      account.withdraw(300);
      try {
        Thread.sleep(400);
      } catch (InterruptedException e) {}
    }
  }
}

public class BankSimulationDemo {
  public static void main(String[] args) {
    BankAccount sharedAccount = new BankAccount();

    DepositThread dt = new DepositThread(sharedAccount);
    WithdrawThread wt = new WithdrawThread(sharedAccount);
```

```
        dt.start();
        wt.start();
    }
}
```

Output:



Conclusion:

I learned how to create and run multiple threads in Java by extending the Thread class to execute tasks simultaneously. I learned about the different lifecycle states of a thread, such as NEW, RUNNABLE, and TERMINATED, and how to control them using methods like start(), sleep(), and join(). Additionally, I learned how to use the synchronized keyword to safely share resources between threads, ensuring they don't interfere with each other during tasks like bank transactions.